UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

INFORME TÉCNICO
LOGIN Y CONEXIÓN A BASE DE DATOS

CUENTA INSTITUCIONAL UPLA Y CUENTA PERSONAL

ALUMNO:

➢ QUISPE TICA RUTH NAYELI

CURSO:

➢ DESARROLLO DE APPS

CICLO:

➢ IV

SECCIÓN:

➢ A1

HUANCAYO - PERÚ

2026

1. Introducción

El presente informe técnico explica el funcionamiento del subsistema de inicio de sesión (login) y la conexión a la base de datos del proyecto web CompuTech. El análisis se basa en los archivos del proyecto Java Web y en el script SQL de CompuTechDB proporcionados para esta actividad.

La aplicación utiliza Java, Jakarta Servlet/JSP y JDBC para comunicarse con Microsoft SQL Server. El proceso de autenticación se distribuye entre la vista login.jsp, el controlador LoginServlet.java, la capa de servicios UsuarioService.java, el acceso a datos UsuarioDAO.java y la clase de conexión Conexion.java. El filtro AuthFilter.java controla el acceso a las rutas de la aplicación.

2. Objetivos

Objetivo general: explicar de forma técnica y ordenada cómo se establece la conexión con CompuTechDB y cómo se autentica un usuario en la aplicación CompuTech.

Objetivos específicos: describir la configuración JDBC; explicar la consulta parametrizada del usuario; analizar el algoritmo PBKDF2 utilizado para verificar contraseñas; detallar la creación y cierre de la sesión HTTP; y evaluar el papel del filtro de seguridad y las restricciones de la base de datos.

3. Arquitectura y tecnologías utilizadas

El proyecto presenta una arquitectura por responsabilidades. La vista JSP muestra el formulario; el Servlet recibe la solicitud HTTP; el servicio aplica las reglas de autenticación y registro; el DAO ejecuta consultas SQL; la clase Conexion crea conexiones JDBC; y SQL Server almacena las cuentas y los datos de la aplicación.

Componente	Responsabilidad en CompuTech
login.jsp	Muestra el formulario y presenta mensajes de error o cierre de sesión.
LoginServlet.java	Procesa /login y /logout, autentica al usuario y gestiona la sesión.
UsuarioService.java	Busca cuentas, verifica contraseñas y valida los datos de registro.
UsuarioDAO.java	Consulta e inserta registros en dbo.Usuario con PreparedStatement.
Conexion.java	Carga la configuración y abre conexiones JDBC con SQL Server.

AuthFilter.java	Permite rutas públicas y exige sesión para las demás rutas.
CompuTechDB	Almacena usuarios, productos, categorías, marcas, proveedores, carrito y pedidos.

4. Configuración de conexión: db.properties

db.host=localhost
db.port=1433
db.name=CompuTechDB
db.user=sa
db.password=123

El archivo se encuentra en src/main/resources/db.properties. db.host define el servidor; db.port indica el puerto de SQL Server; db.name especifica la base de datos; db.user identifica la cuenta de acceso; y db.password contiene la clave de esa cuenta. En el informe se oculta el valor de la contraseña por seguridad.

La clase Conexion.java ofrece valores predeterminados si alguna propiedad no está definida. En el archivo entregado se observan valores predeterminados para el usuario y la contraseña; por ello, antes de desplegar el sistema se debe cambiar cualquier clave de ejemplo, crear una cuenta de base de datos con privilegios mínimos y evitar publicar credenciales reales en repositorios.

La configuración JDBC construida por el programa incluye encrypt=true, trustServerCertificate=true y loginTimeout=8. encrypt=true habilita cifrado de la conexión; trustServerCertificate=true acepta el certificado del servidor sin validarlo plenamente, por lo que es conveniente configurar un certificado confiable en producción; loginTimeout=8 limita el tiempo de espera de conexión a ocho segundos.

5. Explicación de Conexion.java

private static final Properties CFG = new Properties();

Declara el contenedor de propiedades de configuración. Es static para que la configuración se comparta en la clase y final para que la referencia CFG no se reasigne a otro objeto.

try (InputStream in =
Conexion.class.getClassLoader().getResourceAsStream("db.properties")) {
if (in != null) CFG.load(in);
} catch (IOException e) {
// se usan los valores por defecto
}

El ClassLoader busca db.properties dentro de los recursos de la aplicación. Si lo encuentra, CFG.load(in) carga sus valores. El try-with-resources cierra automáticamente

el flujo de lectura. Si ocurre una IOException, el código continúa y utiliza los valores predeterminados definidos en los métodos de configuración.

```
private static String url() {
String host = CFG.getProperty("db.host", "localhost");
String puerto = CFG.getProperty("db.port", "1433");
String bd = CFG.getProperty("db.name", "CompuTechDB");
String servidor = host.contains("\\") ? host : host + ":" + puerto;
return "jdbc:sqlserver://" + servidor + ";databaseName=" + bd
+ ";encrypt=true;trustServerCertificate=true;loginTimeout=8";
}
```

El método url construye la cadena JDBC. Lee el host, el puerto y el nombre de la base de datos. Si el host contiene una barra invertida, interpreta que se ha indicado una instancia con nombre, como una instancia de SQL Server; en ese caso no agrega el puerto de forma separada. La cadena resultante identifica el controlador, el servidor y las opciones de conexión.

```
public static Connection getConexion() throws SQLException {
try {
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
} catch (ClassNotFoundException e) {
throw new SQLException("Falta el driver JDBC de SQL Server (mssql-jdbc).", e);
}
return DriverManager.getConnection(url(),
CFG.getProperty("db.user", "sa"),
CFG.getProperty("db.password", "123"));
}
```

getConexion carga explícitamente el controlador de Microsoft SQL Server. Si no se encuentra la clase del controlador, convierte el problema en una SQLException con un mensaje explicativo. Después llama a DriverManager.getConnection y entrega la URL, el usuario y la contraseña. Si la conexión no puede abrirse, el método comunica el error mediante SQLException.

El método abre una conexión nueva cada vez que se invoca. Los componentes que la usan deben cerrarla; en UsuarioDAO esto se realiza mediante try-with-resources. Para sistemas con muchas solicitudes simultáneas se puede evaluar un pool de conexiones, por ejemplo HikariCP.

6. Acceso a datos: UsuarioDAO.java

```
public Usuario buscar(String login) {
String sql = "SELECT login, clave_hash, nombre, correo, rol "
+ "FROM dbo.Usuario WHERE login = ?";
try (Connection cn = Conexion.getConexion();
PreparedStatement ps = cn.prepareStatement(sql)) {
```

```
ps.setString(1, login.trim());
try (ResultSet rs = ps.executeQuery()) {
if (rs.next()) {
return new Usuario(rs.getString("login"), rs.getString("clave_hash"),
rs.getString("nombre"), rs.getString("correo"), rs.getString("rol"));
}
}
return null;
} catch (SQLException e) {
throw new IllegalStateException("Error de base de datos: " + e.getMessage(), e);
}
}
```

buscar recibe el nombre de usuario login y consulta dbo.Usuario. La consulta solicita únicamente las columnas necesarias para construir el objeto Usuario. El signo ? se reemplaza por el parámetro establecido con ps.setString(1, login.trim()), donde trim elimina espacios al inicio y al final.

executeQuery ejecuta la consulta de lectura y devuelve un ResultSet. Si rs.next() devuelve verdadero, existe una fila y se crea un objeto Usuario con el login, el hash de la contraseña, el nombre, el correo y el rol. Si no hay coincidencia, el método devuelve null. Si ocurre un error SQL, lanza IllegalStateException para que la capa superior pueda tratar el problema.

```
public void insertar(Usuario u) {
String sql = "INSERT INTO dbo.Usuario "
+ "(login, clave_hash, nombre, correo, rol) VALUES (?,?,?,?,?)";
try (Connection cn = Conexion.getConexion();
PreparedStatement ps = cn.prepareStatement(sql)) {
ps.setString(1, u.getLogin());
ps.setString(2, u.getClaveHash());
ps.setString(3, u.getNombre());
ps.setString(4, u.getCorreo());
ps.setString(5, u.getRol());
ps.executeUpdate();
} catch (SQLException e) {
throw new IllegalStateException("Error de base de datos: " + e.getMessage(), e);
}
}
```

insertar registra una cuenta en dbo.Usuario. Cada dato se asigna a un parámetro del PreparedStatement y executeUpdate ejecuta la operación de inserción. Los bloques try-with-resources cierran tanto la conexión como la sentencia, incluso si ocurre una excepción.

El uso de PreparedStatement evita concatenar directamente datos de entrada dentro de la consulta y ayuda a prevenir la inyección SQL. No reemplaza otras medidas de seguridad, como validar datos, limitar privilegios y gestionar mensajes de error.

7. Servicio de autenticación: UsuarioService.java

```
public synchronized Usuario autenticar(String login, String clave) {
if (login == null || clave == null) return null;
Usuario u = buscar(login);
return (u != null && coincide(clave, u.getClaveHash())) ? u : null;
}
```

autenticar comprueba primero que login y clave no sean null. Luego busca el usuario mediante buscar(login). Si el usuario existe y coincide verifica correctamente la contraseña contra el hash almacenado, devuelve el objeto Usuario; en caso contrario, devuelve null. La palabra synchronized hace que las invocaciones de este método sobre la instancia se ejecuten de forma sincronizada, aunque no sustituye las medidas generales de seguridad.

El servicio utiliza una única instancia mediante getInstance(). Además de autenticar, contiene validarRegistro, que revisa nombre, formato de login, correo, longitud de contraseña y coincidencia entre las dos contraseñas del formulario. registrar construye el usuario y lo guarda mediante el DAO con el rol USUARIO, evitando que el formulario público asigne el rol de administrador.

El código verifica si el login ya existe durante la validación, pero una aplicación robusta también debe manejar posibles intentos simultáneos de registro y la restricción de clave primaria de la base de datos. Los mensajes técnicos de conexión deberían registrarse en logs y no exponerse en detalle al usuario final.

8. Protección de contraseñas mediante PBKDF2

```
private static String cifrar(String clave) {
byte[] sal = new byte[16];
new SecureRandom().nextBytes(sal);
return Base64.getEncoder().encodeToString(sal) + ":"
+ Base64.getEncoder().encodeToString(pbkdf2(clave, sal));
}
```

cifrar genera una sal aleatoria de 16 bytes con SecureRandom. Después deriva un hash de la contraseña mediante PBKDF2 y almacena la sal y el resultado en Base64, separados por dos puntos. La sal permite que contraseñas iguales puedan generar resultados distintos.

```
private static byte[] pbkdf2(String clave, byte[] sal) {
PBEKeySpec spec = new PBEKeySpec(clave.toCharArray(), sal, 65536, 256);
return SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")
```

```
.generateSecret(spec).getEncoded();
}
```

PBKDF2WithHmacSHA256 aplica una función de derivación basada en contraseña con HMAC-SHA-256. En este proyecto se usan 65 536 iteraciones y una salida de 256 bits. El resultado es un hash derivado, no la contraseña original.

```
private static boolean coincide(String clave, String guardado) {
String[] partes = guardado.split(":");
byte[] sal = Base64.getDecoder().decode(partes[0]);
byte[] esperado = Base64.getDecoder().decode(partes[1]);
return MessageDigest.isEqual(esperado, pbkdf2(clave, sal));
}
```

coincide separa el texto almacenado en sal y hash, decodifica ambos valores, vuelve a derivar el hash de la contraseña ingresada con la misma sal y compara el resultado con MessageDigest.isEqual. De esta manera, el sistema no necesita guardar la contraseña en texto plano. La implementación presupone que el valor almacenado tiene el formato correcto; para datos corruptos o manipulados conviene validar el formato y gestionar las excepciones.

9. Formulario de acceso: login.jsp

login.jsp es la vista que presenta la interfaz de acceso. Define un formulario con method="post" y action="${ctx}/login", por lo que envía los datos al Servlet asignado a /login. El campo de usuario utiliza name="login" y el campo de contraseña name="clave"; estos nombres coinciden con los parámetros que lee LoginServlet.

```
<form method="post" action="${ctx}/login">
<input type="text" name="login" required>
<input type="password" name="clave" required>
<button type="submit">Ingresar al sistema</button>
</form>
```

El atributo required obliga al navegador a exigir que ambos campos tengan contenido antes del envío, pero la validación del navegador no sustituye la validación del servidor. La página utiliza JSTL para mostrar el mensaje error cuando existe y el aviso de cierre de sesión cuando el parámetro salio vale 1. El contexto ctx permite construir rutas correctas aunque la aplicación se despliegue bajo un nombre de contexto diferente.

10. Controlador LoginServlet.java

LoginServlet está asignado a las rutas /login y /logout. doGet presenta login.jsp cuando se visita /login. Si la ruta es /logout, busca una sesión existente, la invalida si existe y redirige a /login?salio=1.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {
```

```
request.setCharacterEncoding("UTF-8");
Usuario u;
try {
u = usuarioService.autenticar(request.getParameter("login"),
request.getParameter("clave"));
} catch (IllegalStateException e) {
request.setAttribute("error", "No se pudo conectar con la base de datos: " +
e.getMessage());
request.getRequestDispatcher("/login.jsp").forward(request, response);
return;
}
if (u == null) {
request.setAttribute("error", "Usuario o contraseña incorrectos.");
request.getRequestDispatcher("/login.jsp").forward(request, response);
return;
}
HttpSession sesion = request.getSession(true);
sesion.setAttribute("usuario", u);
response.sendRedirect(request.getContextPath() + "/inicio");
}
```

doPost establece UTF-8, obtiene login y clave y llama a UsuarioService.autenticar. Si se produce un error de acceso a datos, prepara un mensaje y reenvía a login.jsp. Si el resultado es null, muestra un mensaje genérico de credenciales incorrectas. Si la autenticación es correcta, crea u obtiene una sesión, almacena el objeto bajo el atributo usuario y redirige a /inicio.

La sesión permite conservar el estado de autenticación entre peticiones HTTP. Como mejora, se recomienda renovar el identificador de sesión tras autenticar, por ejemplo mediante cambio de sesión o una estrategia compatible con el contenedor, para reducir el riesgo de fijación de sesión. También conviene no mostrar al usuario final el detalle interno de la excepción SQL; dicho detalle debe guardarse en los registros de diagnóstico.

11. Filtro de acceso: AuthFilter.java

```
String ruta = request.getRequestURI().substring(request.getContextPath().length());
boolean publico = ruta.equals("/login") || ruta.equals("/login.jsp")
|| ruta.equals("/registro") || ruta.equals("/registro.jsp")
|| ruta.startsWith("/css/") || ruta.startsWith("/js/") || ruta.startsWith("/img/");
HttpSession sesion = request.getSession(false);
boolean autenticado = sesion != null && sesion.getAttribute("usuario") != null;
if (publico || autenticado) {
chain.doFilter(req, res);
} else {
```

response.sendRedirect(request.getContextPath() + "/login");
}

El filtro está mapeado a /*, de modo que intercepta las solicitudes de la aplicación. Calcula la ruta sin el contexto y marca como públicas las páginas de login y registro, además de los recursos estáticos CSS, JavaScript e imágenes. Luego comprueba si existe una sesión con el atributo usuario.

Si la ruta es pública o existe una sesión autenticada, chain.doFilter permite que la petición continúe. En caso contrario, redirige al usuario a /login. Este control centraliza la verificación de sesión y evita repetirla en cada Servlet.

Limitación importante: el filtro comprueba que exista el atributo usuario, pero no verifica por sí solo los permisos del rol ADMIN, ALMACENERO o USUARIO. Las operaciones restringidas deben comprobar autorización por rol en los controladores o servicios correspondientes. Además, si se añaden nuevas rutas públicas, deberán incorporarse explícitamente al filtro.

12. Estructura de la base de datos CompuTechDB

El script SQL suministrado crea la base de datos CompuTechDB y define ocho tablas principales: Categoria, Marca, Proveedor, Producto, Usuario, Carrito, Pedido y DetallePedido. La tabla dbo.Usuario contiene login como clave primaria, clave_hash, nombre, correo y rol. Una restricción CHECK limita el rol a ADMIN, ALMACENERO o USUARIO.

La tabla Usuario guarda el hash en la columna clave_hash y no una contraseña en texto plano. El script incluye dos cuentas iniciales con los roles ADMIN y ALMACENERO. Las cuentas y contraseñas de ejemplo deben cambiarse en la instalación real y tratarse como datos sensibles.

Advertencia sobre el script: al inicio, el archivo SQL comprueba si CompuTechDB existe y, en ese caso, la elimina y la vuelve a crear. Ejecutarlo nuevamente puede borrar datos existentes. Debe utilizarse únicamente cuando se quiera reinicializar la base de datos y después de realizar una copia de seguridad.

13. Flujo de ejecución completo

Paso	Proceso
1	El usuario abre login.jsp y completa login y clave.
2	El formulario envía una solicitud POST a /login.
3	LoginServlet lee los parámetros y llama a UsuarioService.autenticar.

4	UsuarioService solicita al UsuarioDAO buscar el login.
5	UsuarioDAO obtiene una conexión de Conexion.java y consulta dbo.Usuario.
6	UsuarioService compara la clave ingresada con el hash PBKDF2 almacenado.
7A	Si coincide, LoginServlet guarda el objeto Usuario en la sesión y redirige a /inicio.
7B	Si no coincide, vuelve a login.jsp con un mensaje de error.
8	AuthFilter permite las rutas públicas y exige sesión para el resto de páginas.
9	Al cerrar sesión, /logout invalida la sesión y redirige al login.

14. Pruebas recomendadas

Prueba	Resultado esperado
Conexión válida y credenciales correctas	Se crea la sesión y se accede a /inicio.
Contraseña incorrecta	Se muestra “Usuario o contraseña incorrectos.” y no se crea una sesión autenticada.
Usuario inexistente	Se muestra el mismo mensaje genérico de credenciales incorrectas.
SQL Server detenido o conexión mal configurada	La aplicación presenta un aviso de conexión fallida; el detalle debe quedar en logs.
Acceso directo a una ruta privada sin sesión	AuthFilter redirige a /login.
Acceso a /css/ o /img/ sin sesión	Los recursos públicos pueden cargarse.
Cerrar sesión	La sesión se invalida y se vuelve al login.
Registro de nuevo usuario	La contraseña se almacena como PBKDF2 con sal y el rol asignado es USUARIO.

15. Evaluación y recomendaciones de seguridad

Fortalezas observadas: separación entre vista, controlador, servicio y DAO; consultas parametrizadas; cierre automático de recursos JDBC; uso de sal aleatoria y PBKDF2 para contraseñas; mensajes genéricos ante credenciales incorrectas; y filtro centralizado para exigir sesión.

Recomendaciones: cambiar las credenciales predeterminadas y limitar los permisos de la cuenta SQL; no publicar db.properties con secretos; validar certificados TLS en producción en lugar de confiar automáticamente en cualquier certificado; renovar el identificador de sesión después de iniciar sesión; verificar permisos por rol en cada operación sensible; registrar errores sin revelar detalles internos en la interfaz; añadir límites o demoras frente a intentos repetidos; y revisar el formato del hash antes de decodificarlo.

También se recomienda comprobar que el controlador JDBC de Microsoft SQL Server esté disponible como dependencia del proyecto, que SQL Server acepte conexiones TCP/IP en el host y puerto configurados y que el usuario tenga permisos sobre CompuTechDB. La conexión debe probarse antes de evaluar el login para distinguir errores de infraestructura de errores de autenticación.

16. Conclusiones

El login de CompuTech se apoya en un flujo por capas: la vista envía las credenciales, el Servlet coordina la solicitud, el servicio valida la contraseña mediante PBKDF2, el DAO consulta la tabla dbo.Usuario y Conexion.java establece la comunicación JDBC con SQL Server. Cuando las credenciales son correctas, se almacena el usuario en la sesión; si no lo son, se devuelve un mensaje de error. AuthFilter protege las rutas que requieren autenticación.

La implementación incluye prácticas positivas como PreparedStatement, cierre de recursos, sal aleatoria y derivación PBKDF2. Aun así, para un despliegue real es necesario reforzar la administración de credenciales, la validación del certificado TLS, la renovación de sesiones y la autorización basada en roles. Finalmente, debe tenerse especial cuidado con el script SQL, porque está diseñado para eliminar y reconstruir CompuTechDB si ya existe.